Acquiring Data in React with useState

Now we are going to get our React page to interact by making API calls to server. When last we left our React page in *Dynamic styled-components*, it was displaying the seating information for one specific concert. What we'd like it to do now is get that seating information from the server and update it if it changes. This will use the fetch method we've already seen, as well as a new React hook called useEffect.

Updating the seating information involves adding the following features to the page:

- When the React components load, they need to get the current seating status from the server.
- That status needs to get passed down from the Venue component to the seat component that uses the status to display. In this chapter, we're going to keep the status in our component; in Chapter 11, *Managing State in React*, we'll look at how we can keep the status in a central global repository.
- We need to make sure a user can't adjust a seat that is already sold. (This is a UI bit we didn't do when we were working on this page earlier.)
- When the user does click on a seat, adding it to the shopping cart, we want the page to update the server so that any other user looking at the page sees the updated status.

That's a lot, but most of it is similar to React features we've already used. Note that we don't need to explicitly handle authentication, since our React code is not an SPA; the Rails server is handling it and will handle it in the Rails session on our API calls.

Let's look at the new React parts first. Once that's done, we need to add a couple of new Rails controller actions, but we'll talk about them after we see how the React code works. We have a hierarchy of components here—Venue to VenueBody